Team Rayquaza: Transformer Fleet Health & Replacement Planning

HCLTech Future Ready AI Engineer Hackathon · DS/ML edition · 11 Oct 2026 · mentor brief

WHAT WE UNDERSTOOD

A utility has about 400 transformers and money to fix only a few. We predict each one's **chance of failing in the next 12 months**, calibrated so that "20%" really means about 20%, with an uncertainty range. We compute **risk = probability × impact** (hospital 5, industry 3, urban 2, rural 1, × customers), then choose replace / inspect actions within the budget using a **knapsack**, and show it beats an age-based plan. Units with no oil test are never called "healthy".

OUR PLAN

1 Generate data (seeded, edge cases planted, data card) → **2 Load + validate** (HCL checks as asserts) → **3 Clean** (unit jumps, duplicates, gaps) → **4 Snapshot table**: one row = one transformer at one quarterly snapshot, only past data, label = fails in next 12 months → **5 Features** (gas ratios and trends, load avg/peak, hot months, age, months since test, *has_oil_test*; one function shared by training and the API) → **6 Baselines** (age-only, rule-based health index) → **7 Models**: logistic regression and gradient boosting × **3 missing-data methods** (median, median + indicator, native) → **8 Validation**: time-aware and grouped by transformer, repeated 5×, CIs; final held-out snapshots used once → **9 Calibrate** (Platt: suits few positives) + bootstrap intervals → **10 Risk + knapsack plan** vs age plan → **11 Serve**: FastAPI (/predict /plan /ingest /model/info /explain /health) + Streamlit + Docker compose, experiment log, model card, tests

KEY DECISIONS (SO FAR)

- **Simple first:** logistic regression is the expected winner with few failures; boosting has to earn its place.
- **scikit-learn only**: no extra installs; native NaN support covers missing-data method 3.
- **Platt over isotonic** calibration: isotonic overfits with ~30–50 positives.
- **Missingness is a feature:** *has_oil_test*, because untested may itself mean risk.
- **SQLite** store; snapshot features from as-of joins (window functions).

DATA: WE GENERATE ALL 4 FILES

- 400 transformers, 36 months (Oct 2023 – Sep 2026), ~38% with no oil test.
- Failures from a documented hazard: age, overload, developing gas fault.
- All 6 HCL edge cases planted on purpose; IDs in the data card.
- **Extra columns:** *rating_class* (customers_served), *event* = failure / repair (operation).
- Pipeline must still run on judges' files with **only** Annex B columns.

QUESTIONS FOR THE MENTORS — WRITE ANSWERS VERBATIM

1. **We are generating all data ourselves.** Can failure events be an **extra column** (*event*) in operation.csv, and size class an extra column in customers_served.csv? Or do you prefer a separate file?

2. On judging day, will **your test CSVs contain only Annex B columns**? If *rating_class* is absent, which cost should a plan use?

3. How are we scored on the **hidden outcomes**: your data loaded through our loader? Which **as_of / snapshot date** counts as "now"?

4. What risk does an **inspection** remove compared with a **replacement**? (The knapsack needs a value per action. Default: replace removes all of it, inspect a stated fraction.)

5. Budget for **/plan**: from *annual_budget* in the file, or the request body? Does the late-drop budget come through **/ingest**?

6. Is **customers_served.csv** acceptable as the "asset list" in the validation checks?

7. Which metric matters most on hidden outcomes (PR-AUC / top-10% recall / Brier)? Is judging the **30-min slot** (§9.1) or **5 + 5 min** (schedule)?

Our assumptions where unanswered go into the README's "Assumptions" section and our decision log.